SunXFinder Website Launch Readiness Report

Audit date: 28 October 2026

Scope: Source repository, local runtime configuration, Laravel routes, existing tests, asset build, and visible operational code

Decision: NO-GO until the production blockers below are verified or resolved.

Executive summary

The repository contains a substantial Laravel 12 marketplace with customer registration, company accounts, a customer API, admin tools, and mobile-app integration. The number of planned accounts - 500 customers and 100 companies - is modest for a relational database. Account count alone does not prove that the site will be responsive: concurrency, API request patterns, image traffic, database queries, and server limits determine capacity.

This audit cannot confirm that the live production system is ready to accept customers. The checked-out environment reports **local**, debug enabled, and URL **127.0.0.1**; the production environment, host, database, mail provider, and monitoring were not available for inspection. The browser check could not fetch either **sunxfinder.com** or **dev.sunxfinder.com**; that is an audit-tool limitation, not evidence that either site is down.

The current checkout also has an incomplete frontend build, a failing feature test, no npm lockfile, no visible CI/deployment workflow, and no configured external uptime monitoring or scheduled off-site backup in the repository. These are launch gates because the site cannot be treated as production-ready or as having reliable recovery until they are addressed.

No system can guarantee that a website will **never** go offline. A realistic goal is measurable availability, rapid detection, and recovery through redundant hosting, tested backups, and alerts.

Findings by priority

P0 - production configuration is unverified

php artisan about --only=environment reports:

- Laravel 12.69.3 and PHP 8.2.0 in the local checkout.
- Environment: **local**.
- Debug mode: enabled.
- URL: **127.0.0.1**.

These values are appropriate for local development, not evidence of a safe production deployment. Confirm the actual production host has **APP_ENV=production**, **APP_DEBUG=false**, the canonical HTTPS **APP_URL**, a non-empty protected **APP_KEY**, correct production database credentials, secure session cookies behind TLS, and working mail and Firebase settings. Do not copy **.env.example** unchanged: it contains development defaults (**APP_DEBUG=true**, **APP_URL=http://localhost**, SQLite, and log mail).

The project root contains an ignored Firebase service-account JSON file. Keep credentials outside the public web root and out of Git/deployment artifacts unless the application specifically loads them from a protected location. Ensure the web server document root is Laravel's **public/** directory, never the repository root. Laravel's deployment guide also requires PHP 8.2 or newer and write access to **storage** and **bootstrap/cache**.

P0 - the production asset build is missing in this checkout

resources/views/visitors/welcome.blade.php uses Laravel Vite. **public/build/manifest.json** is absent, and **npm run build** fails because **vite** is not installed in this checkout. There is a **package.json** but no **package-lock.json**, so the JavaScript dependency graph is not locked for repeatable releases.

Before launch, create and commit the appropriate npm lockfile, install from that lockfile in the release pipeline, run the production build, deploy **public/build/manifest.json** and its referenced assets, and verify that the home page and every generated asset return successfully. A release without the Vite manifest can fail while rendering the visitor home page.

P0 - the existing feature test fails

php artisan test result: **1 passed, 1 failed**. The feature test **tests/Feature/ExampleTest.php** requests **/**, but receives HTTP 500 because the in-memory SQLite test database has no **tbl_settings** table. The unit test passes, but the current test suite does not provide a reliable launch regression check. Add migration-aware test setup and focused tests for the home page, customer/company registration, login, core API flows, and admin protection.

P1 - API capacity and response time have not been measured

The API routes use Sanctum on protected customer endpoints and include request throttles (commonly 60 requests/minute per authenticated user, 120/minute for API configuration, and 5/minute on login/registration). These limits constrain callers; they do not establish the server's throughput or response time.

There is no production-like load-test result, p95/p99 latency baseline, error-rate baseline, database query profile, or documented server resource limit in the repository. Before accepting a large launch campaign, load-test a staging environment using realistic app flows: startup/main data, company listings and details, quote creation, offers, notifications, and registration/login. Measure latency, 5xx/timeouts, CPU, memory, database connections, and queue backlog under expected and burst traffic. Set a capacity limit based on the actual host and test results.

P1 - uptime monitoring and alerting are not configured here

Laravel exposes the standard **/up** health route, but no external uptime monitor, alert destination, or incident procedure is configured in the repository. Monitoring should check the public home page and **/up** from outside the hosting provider and alert a person who can respond. Track application 5xx rates, database availability, disk space, queue backlog, and mail/push failures as well.

Choose an availability target with the hosting provider. For example, a 99.9% monthly target permits roughly 44 minutes of downtime in a 30-day month; it is a service objective, not a promise of zero downtime. A single shared server or database remains a single point of failure.

P1 - database backups are manual and restore is unverified

The repository provides an admin-triggered database export stored under **storage/app/private/db**. The scheduler contains only daily activity-log pruning; no automated application backup job is registered. Backups stored on the same host do not protect against host or account loss.

Configure automated encrypted off-site database and uploaded-file backups, retention, failure alerts, and a documented restore procedure. Perform a restore test before launch and on a regular schedule. Define recovery targets (RPO: acceptable data loss; RTO: acceptable recovery time) and make sure they match the business need. The hosting provider may offer separate backup service, but that was not verifiable in this audit.

P1 - queue and scheduled-task operation needs a production plan

The example environment uses the database queue. If production uses that queue, background jobs will remain pending unless a worker runs continuously and restarts after failure. Laravel's local Composer **dev** script starts a queue listener, but no production process manager or deployment workflow is present. The scheduler currently runs **model:prune** daily; production also needs a cron entry that invokes **schedule:run** every minute.

Confirm which production tasks are queued, configure a supervised worker for the selected queue driver, monitor failed jobs/backlog, and configure the scheduler. If the host cannot run persistent workers, move those tasks to a supported managed queue or synchronous alternative with known request-time impact. Laravel documents queue workers as persistent processes that need a process monitor.

P1 - dependency vulnerability status is unknown

composer validate --no-check-publish passes. **composer audit --locked** could not reach Packagist because DNS/network access timed out, so dependency vulnerability status is **inconclusive**, not clean. Repeat the audit in CI or another network-enabled environment before release, and add dependency update/security checks to the release process. The JavaScript dependencies also need a committed lockfile and an audit.

P2 - public account-creation routes lack explicit rate limits

The web POST routes for customer and company account creation have validation but no route-level throttle in **routes/web.php**. The contact form is throttled and has a honeypot, and API login/registration are throttled. Add reasonable rate limits and abuse monitoring to the public web signup routes, especially because company registration accepts an uploaded logo.

P2 - local test tools should remain strictly local

/push, **/email-en**, and **/email-ar** are wrapped in **if (app()->isLocal())**, which is a useful guard. The test push method contains a hard-coded Firebase device registration token. Remove the test route/token from release code or load a dedicated test token from protected local configuration, and rotate the token if it is real. Production must report **APP_ENV=production** so local-only routes are not registered.

P2 - cache clearing uses GET

GET /clear invokes **optimize:clear** and is protected by **admin.auth**, but it changes application state through a GET request. Convert it to a CSRF-protected POST action or a deployment/console operation to prevent accidental execution from navigation or link previews.

Positive controls found

- Protected customer API routes use **auth:sanctum**; admin and company panels have authentication middleware.
- Login, password reset, contact form, API configuration, and most API endpoints have throttles.
- Customer and company account creation validates submitted fields; the contact form uses a honeypot.
- The custom database backup code writes under a private storage path and the backup directory is excluded from Git.
- The Apache public **.htaccess** routes requests through Laravel, disables directory listing, compresses text responses, and caches static assets with versioned URLs.
- **composer.lock** exists, **composer validate** passes, and **php artisan route:list** successfully loads 188 routes.

Checks performed

Check	Result
php artisan about --only=environment	Local environment; debug enabled; URL **127.0.0.1**; PHP 8.2.0
composer validate --no-check-publish	Passed
php artisan route:list --except-vendor	Passed; 188 routes loaded
php artisan test	1 passed, 1 failed; feature test database missing **tbl_settings**
npm run build	Failed; **vite** executable unavailable

Check	Result
Vite manifest check	**public/build/manifest.json** absent; visitor home references **@vite**
php artisan schedule:list	Only activity-log pruning is scheduled
composer audit --locked	Inconclusive; Packagist DNS/network timeout
Live **sunxfinder.com** / **dev.sunxfinder.com** fetch	Not available from this audit environment; no live uptime conclusion

Launch gate checklist

Do not announce production readiness until all P0 items are complete and the owner can verify these items in the deployed environment:

1. Production settings are confirmed (**APP_ENV=production**, **APP_DEBUG=false**, HTTPS canonical URL, valid app key, correct DB, mail, and Firebase credentials).
2. The host meets Laravel 12's PHP and extension requirements, serves only Laravel **public/**, and grants correct storage/cache permissions.
3. A reproducible Vite build is produced and the deployed home page, CSS, JavaScript, images, and web/mobile deep links work.
4. Database migrations and storage paths are applied; customer signup, company signup, login, password reset, quote flows, email, and push notifications are smoke-tested with real production integrations.
5. Automated off-site backups are running and a restore has been demonstrated.
6. A queue worker and scheduler are configured and monitored if used.
7. External uptime alerts, error reporting, a support owner, and a rollback procedure are in place.
8. A staging load test confirms acceptable response times and error rates at expected peak concurrency and a chosen safety margin.
9. Composer and npm dependency audits complete successfully on a network-enabled release runner.
10. Local debug/test endpoints and test tokens are absent or safely disabled in production.

Bottom line

The codebase is not yet verified as ready to accept production customers. The 500-customer/100-company account count is not the primary concern; unverified production configuration, the missing frontend build, the failing feature test, and absent demonstrated monitoring/recovery/load capacity are. Complete the release gates, then reassess using the actual production host and measured load. No evidence in this repository can support a claim that the site will never be offline.

References

- Project code: **routes/api.php**, **routes/web.php**, **routes/console.php**, **resources/views/visitors/welcome.blade.php**, **tests/Feature/ExampleTest.php**, **phpunit.xml**, **.env.example**, **app/Support/DatabaseBackup.php**, **app/Http/Controllers/WebsiteController.php**, **config/queue.php**, **public/.htaccess**.
- Laravel 12 deployment guide
- Laravel 12 queues guide